

Retrofitting Runtime Security onto Legacy Embedded Linux with Modular eBPF Supervisors

Alberto Scala

Abstract

Many embedded Linux devices run a firmware which will never be rebuilt and which lacks, in large part, the standard mitigations against memory corruption. In this work I investigate if a runtime protection can be added to such a device without modifying its firmware. After discarding a virtual-machine design, too heavy for old hardware, and a user-space supervisor, too slow to intercept system calls, I adopt eBPF in order to load small, verified programs into the kernel already present on the device. I present a modular architecture of independent supervisors, following the so-called observe-then-enforce approach, and two supervisors: TOLL, which restricts IPv4 traffic to whitelisted peers in both directions, and GUARDIAN, which uses the BPF LSM hooks to deny executable memory that's or was writable and to allow the execution of whitelisted binaries only. Functional tests on a workstation show the intended decisions in the nominal cases and the absence of false positives with ordinary programs; the evaluation on embedded hardware and against real exploits is left as future work.

1 Introduction

Routers, gateways, industrial controllers and cameras run, very often, a firmware built once and never again: the toolchain is lost, the producer has ended the support, and reflashing is risky. In their work, Yu et al. [1] observed that user-space and kernel mitigations are largely absent in more than 10,000 Linux firmware images, and that their adoption isn't improving. All the classical remedies (canaries, `_FORTIFY_SOURCE`, PIE, RELRO, a newer kernel) require a rebuild. In addition, starting from September 2026 the EU Cyber Resilience Act obliges manufacturers to report actively exploited vulnerabilities within 24 hours [2]; it becomes therefore necessary to see the exploitation, and to stop it, before a fix exists.

The aim of this work is to understand if such a protection can be added *without modifying the firmware*. In particular, the contributions are the following: (i) an analysis of two previous designs and of the reasons for which I abandoned them; (ii) a modular architecture of independent eBPF supervisors with an observe-then-enforce rollout; (iii) TOLL, a bidirectional network supervisor; and (iv) GUARDIAN, a memory and exec supervisor based on the BPF LSM [3], together with a functional evaluation which states what it doesn't cover.

2 Background and gap

Build-time mitigations are effective but are properties of the binary, and binary rewriters, too, modify and reflash every program. Kernel access control, on the other hand, can block the final step of an exploit: SELinux [4] denies writable executable memory and restricts execution, and IMA [5] appraises binaries. However, they need a

system-wide policy for an undocumented firmware or a labeled filesystem, while `seccomp` [6] and `PR_SET_MDWE` are opt-in per process. Isolation by means of containers or hypervisors limits the damage but doesn't stop the exploit, and often needs hardware support that old devices lack. For what concerns eBPF, the existing tools avoid the rebuild, but are either detection-only (Falco [7]), general-purpose agents conceived for cloud hosts (Tetragon [8]), or confinement systems requiring a per-program policy (bpfbox [9]); moreover, LSM-based enforcement and embedded targets are still underexplored [10]. What is missing, in the end, is a small, mandatory, in-kernel enforcement which needs no change to the user space of the device.

3 Why eBPF

In my first design the containerized firmware ran in a virtual machine; this resulted too heavy for boards without virtualization support and, in any case, the exploit could still run inside the guest. My second design, instead, placed a privileged supervisor in user space; it was sufficiently light, but every intercepted `mmap`, `mprotect` or packet required a round trip out of the kernel. From these attempts I derived three requirements: no extra kernel or hypervisor, no firmware change, and decisions taken in the kernel before the operation takes effect. eBPF satisfies all three: verified programs run at the hook itself, keep their state in kernel maps, and can be loaded and removed at runtime. The price is a kernel floor of `bpf()`, BTF and, for GUARDIAN, BPF LSM (Linux 5.7+).

4 Architecture

Each supervisor faces one single threat and is composed by an independent pair: eBPF programs in the kernel and a small loader in user space, both in C with `libbpf` and CO-RE. The loader opens a self-contained skeleton, lets the kernel verify the programs, writes the policy into their global data and, only at the end, attaches them, in such a way that no event is ever evaluated against an empty whitelist. The supervisors share no code or runtime dependency. At the moment, the sharing of state through pinned maps and task-local storage, coordinated by a daemon, is designed but not implemented yet. Since a wrong policy on an embedded device fails silently, every supervisor is meant to run first in *log-only* mode, reporting what it would deny; the observed behaviour then becomes the policy, and enforcement is enabled.

5 Toll: network

A compromised device is dangerous above all through the network; nevertheless, a fixed-function device talks with few, stable peers. TOLL is attached to the TC ingress and egress hooks, the only packet-level hook that sees both directions with one program type (XDP,

in fact, sees only ingress). It checks the source address of incoming packets and the destination of outgoing ones against two separate IPv4 whitelists, so that the policy can be asymmetric, and drops all the other IPv4 packets with `TC_ACT_SHOT`. Non-IPv4 frames pass; in this way ARP keeps working. The whitelists are bounded global arrays filled by the loader; any error while reading them aborts before attaching.

6 Guardian: memory and exec

GUARDIAN addresses a common chain: a buffer overflow hijacks a return address, a so-called ROP chain, built from existing code, [11] runs, and the chain ends by calling `mprotect/mmap` to make written memory executable, or `execve` (directly or through `system()`) to start a shell. The first three steps involve no system call and are invisible to eBPF; the last one, on the contrary, must pass through the kernel, and a BPF LSM hook can refuse it with `-EPERM` before it takes effect.

Memory. Legitimate code is mapped from files and made executable without ever being writable, whereas shellcode always lies in memory that’s or was writable. For this reason GUARDIAN denies execute permission on memory that’s writable now, or *has ever been* writable, even if the final `mprotect` asks only for `PROT_EXEC`. The history is a hash map keyed by $(mm, page)$, that’s, the address space and a page-aligned address; in this way processes are kept apart and the check doesn’t depend on how the kernel splits or merges its VMAs. Since a per-process history cannot see a write through one mapping of a shared object and an execute through another, execute permission is denied also on all shared mappings. The hooks are `mmap_file`, `mmap_addr` and `file_mprotect`.

Exec. At `bprm_check_security`, execution is denied by default and allowed only if the inode of the file opened by the kernel is in the whitelist. The check is on the target binary, not on the caller, so it covers also `system()`; matching by inode defeats path spoofing and symbolic links, but must be regenerated after firmware updates.

7 Evaluation

The evaluation is of functional type, and not a security or performance study; it was conducted on a development workstation. A standard-library Python script tests the same loader binary that would be deployed (Table 1). The programs of TOLL are run on hand-built packets with `BPF_PROG_TEST_RUN`, then attached inside two network namespaces joined by a veth pair and exercised with real traffic. GUARDIAN, which has no test-run support, is attached to the live kernel in log-only mode while ordinary workloads run: shell commands, Python loading native libraries, and a writable anonymous mapping, which together exercise library loading and the RELRO `mprotect` calls of the dynamic loader.

All the checks passed with success. It is important to underline, however, that the tests do not cover GUARDIAN in enforcing mode, the deny path of the memory hooks, a real exploit chain, IPv6, embedded

Method	Checks	Result
TOLL, unit	allow/drop per direction; ARP untouched	5/5
TOLL, netns	real traffic both ways; re-stored on detach	6/6
GUARDIAN, log-only	exec logged; 0 false positives on 3 memory hooks	7/7

Table 1: Functional checks on a development workstation.

hardware, or any overhead measurement.

8 Limitations and conclusion

GUARDIAN acts only at the last step: attacks which remain within existing code are not stopped, code written to disk and then mapped is never writable *in memory*, and writes through `/proc/pid/mem` or `ptrace` bypass the monitored hooks. The prototype doesn’t record yet the write permission granted at `mmap` time, checks only 64 pages per region, and never evicts history entries. For what concerns TOLL, it passes IPv6 without filtering and has no log-only mode. Finally, a root attacker can unload both supervisors, and the kernel floor excludes the oldest devices.

In conclusion, within these limits, the results suggest that eBPF represents a viable way to add runtime protection to embedded Linux devices after deployment, with no rebuild and no change to their user space. As future developments, I intend to close the known gaps, to run GUARDIAN in enforcing mode, and to evaluate both supervisors on real hardware against real exploits, measuring the overhead and the effort needed to build a whitelist for an undocumented firmware.

References

- [1] R. Yu, F. Del Nin, et al., “Building Embedded Systems Like It’s 1996,” NDSS, 2022.
- [2] European Commission, “Cyber Resilience Act – Reporting obligations.”
- [3] Linux kernel documentation, “LSM BPF Programs.”
- [4] P. Loscocco and S. Smalley, “Integrating Flexible Support for Security Policies into the Linux Operating System,” USENIX ATC (FREENIX), 2001.
- [5] R. Sailer et al., “Design and Implementation of a TCG-based Integrity Measurement Architecture,” USENIX Security, 2004.
- [6] Linux kernel documentation, “Seccomp BPF.”
- [7] Falco, cloud-native runtime security. <https://falco.org/>
- [8] Tetragon. <https://tetragon.io/>
- [9] W. Findlay, A. Somayaji, D. Barrera, “bpffbox: Simple Precise Process Confinement with eBPF,” ACM CCSW, 2020.
- [10] Kostopoulos, Tsakonas, Markakis, “eBPF-Based Cybersecurity Mechanisms: A Systematic Literature Review,” IJIS, 2026.
- [11] H. Shacham, “The Geometry of Innocent Flesh on the Bone,” ACM CCS, 2007.